Kolumnowe bazy danych cz. I

Eksploracja danych, podstawowe agregacje i pierwszy benchmark PostgreSQL i ClickHouse

---

Imiona i nazwiska: Karolina Węgrzyn, Patrycja Markiewicz

Parametry sprzętów:

- zadania parzyste - 8 rdzeni i 32 GB RAM
- zadania nieparzyste - 8 rdzeni i 16 GB RAM

---

1. Pierwsze poznanie tabeli events

PostgreSQL

```
select column_name, data_type, is_nullable, column_default
from information_schema.columns
where table_name = 'events';
```

Output | ds_lab.information_schema.columns

	column_name	data_type	is_nullable	column_default
1	event_time	timestamp without time …	YES	<null>
2	user_id	integer	YES	<null>
3	session_id	integer	YES	<null>
4	product_id	integer	YES	<null>
5	price	double precision	YES	<null>
6	quantity	integer	YES	<null>
7	country	text	YES	<null>
8	device	text	YES	<null>
9	event_type	text	YES	<null>

```
select * from events limit 10;
```

Output ds_lab.public.events

	event_time	user_id	session_id	product_id	price	quantity	country	device	event_type
1	2025-05-05 03:56:41.000000	7297	6557	4507	126.22	1	DE	tablet	view
2	2025-04-16 21:35:32.000000	5698	154795	6913	20.73	1	IT	tablet	view
3	2025-04-19 23:35:29.000000	13032	187701	8929	212.66	1	ES	mobile	view
4	2025-02-01 00:03:58.000000	45754	110786	5575	142.55	1	US	mobile	add_to_cart
5	2025-01-19 18:43:51.000000	23527	90166	9892	135.94	1	SE	tablet	add_to_cart
6	2025-06-29 02:12:28.000000	24888	20657	9045	150.12	1	US	tablet	add_to_cart
7	2025-01-14 12:09:01.000000	3004	173347	3734	387.67	3	DE	desktop	view
8	2025-05-04 10:30:15.000000	23910	42639	6066	180.86	1	DE	tablet	view
9	2025-04-14 17:16:20.000000	47785	64176	2678	233.82	1	NO	mobile	view
10	2025-06-13 15:58:28.000000	3666	60044	526	403.5	1	DE	mobile	view

10 rows

```sql
SELECT
    count(*) AS n,
    min(event_time) AS min_time,
    max(event_time) AS max_time
FROM events;
```

Output Result 18

	n	min_time	max_time
1	1000000	2025-01-01 00:00:02.000000	2025-06-29 23:59:25.000000

```sql
SELECT
    count(*) AS all_rows,
    count(price) AS non_null_price,
    count(quantity) AS non_null_quantity,
    count(*) - count(price) AS null_price_rows,
    count(*) - count(quantity) AS null_quantity_rows
FROM events;
```

Output Result 19

	all_rows	non_null_price	non_null_quantity	null_price_rows	null_quantity_rows
1	1000000	1000000	1000000	0	0

ClickHouse

```sql
DESCRIBE TABLE events;
```

	name	type	default_type	default_expression	comment	codec_expression
1	event_time	DateTime				
2	user_id	UInt32				
3	session_id	UInt32				
4	product_id	UInt32				
5	price	Float64				
6	quantity	UInt32				
7	country	String				
8	device	String				
9	event_type	String				

Pozaostałe zapytanie były takie same dla ClickHouse jak dla PostgreSQL.

Output | ds_lab.events | ds_lab.information_schema.columns | Result 1

	event_time	user_id	session_id	product_id	price	quantity	country	device	event_type
1	2025-01-01 00:00:02	28501	120283	6168	313.69	1	PL	mobile	view
2	2025-01-01 00:00:08	23027	122020	8183	335.26	1	DE	tablet	view
3	2025-01-01 00:00:11	16049	199024	3590	50.87	1	ES	tablet	view
4	2025-01-01 00:00:13	32566	163522	214	398.55	5	ES	desktop	view
5	2025-01-01 00:01:02	19380	98783	556	39.42	1	SE	tablet	view
6	2025-01-01 00:01:36	20025	98269	4925	134.53	4	US	desktop	view
7	2025-01-01 00:01:41	22550	133901	1248	335.74	1	IT	tablet	view
8	2025-01-01 00:01:59	13356	54446	7423	306.36	1	PL	desktop	view
9	2025-01-01 00:02:16	7098	44757	4338	416.66	1	ES	desktop	view
10	2025-01-01 00:02:21	3430	83474	6076	452.23	5	GB	desktop	view

Output | ds_lab.events | count(column_name):UInt64 | Result 4

	n	min_time	max_time
1	1000000	2025-01-01 00:00:02	2025-06-29 23:59:25

Output | Result 5 | count(column_name):UInt64 | Result 4

	all_rows	non_null_price	non_null_quantity	null_price_rows	null_quantity_rows
1	1000000	1000000	1000000	0	0

Dla obu baz sprawdzane elementy są takie same - kolumny tabel, liczba rekordów, zakres czasu, brak wartości NULL dla kolumn price i quantity.

2. Profil danych zdarzeniowych

ClickHouse

Sprawdź, jakie typy zdarzeń występują w danych i ile ich jest:

```sql
select
    event_type,
    count(*) as n
from events
group by event_type
order by n desc;
```

	event_type	n
1	view	799261
2	add_to_cart	150406
3	purchase	50333

Sprawdź, jakie kraje występują w danych i ile mają zdarzeń:

```sql
select
    country,
    count(*) as n
from events
group by country
order by n desc;
```

	country	n
1	FR	100290
2	DE	100220
3	US	100193
4	NL	100061
5	ES	100022
6	NO	99972
7	PL	99954
8	GB	99893
9	IT	99734
10	SE	99661

Sprawdź, jakie urządzenia występują w danych i ile mają zdarzeń:

```
select
    device,
    count(*) as n
from events
group by device
order by n desc;
```

	device	n
1	mobile	333764
2	desktop	333355
3	tablet	332881

Dla każdego z tych trzech przekrojów wskaż kategorię dominującą:

```
(select
     'event_type' as przekroj,
     event_type as dominujaca_wartosc,
     count(*) as n
from events
group by event_type
order by n desc limit 1)

UNION ALL

(select
     'country' as przekroj,
     country as dominujaca_wartosc,
     count(*) as n
from events
group by country
order by n desc limit 1)

UNION ALL

(select
     'device' as przekroj,
     device as dominujaca_wartosc,
     count(*) as n
from events
group by device
order by n desc limit 1);
```

	przekroj	dominujaca_wartosc	n
1	country	FR	100290
2	device	mobile	333764
3	event_type	view	799261

PostgreSQL

Sprawdź, jakie typy zdarzeń występują w danych i ile ich jest:

```
select
    event_type,
    count(*) as n
from events
group by event_type
order by n desc;
```

	event_type	n
1	view	799261
2	add_to_cart	150406
3	purchase	50333

Sprawdź, jakie kraje występują w danych i ile mają zdarzeń:

```
select
    country,
    count(*) as n
from events
group by country
order by n desc;
```

	country	n
1	FR	100290
2	DE	100220
3	US	100193
4	NL	100061
5	ES	100022
6	NO	99972
7	PL	99954
8	GB	99893
9	IT	99734
10	SE	99661

Sprawdź, jakie urządzenia występują w danych i ile mają zdarzeń:

```
select
    device,
    count(*) as n
from events
group by device
order by n desc;
```

	device	n
1	mobile	333764
2	desktop	333355
3	tablet	332881

Dla każdego z tych trzech przekrojów wskaż kategorię dominującą:

```
(select
     'event_type' as przekroj,
     event_type as dominujaca_wartosc,
     count(*) as n
from events
```

```
group by event_type
order by n desc limit 1)

UNION ALL

(select
     'country' as przekroj,
     country as dominujaca_wartosc,
     count(*) as n
from events
group by country
order by n desc limit 1)

UNION ALL

(select
     'device' as przekroj,
     device as dominujaca_wartosc,
     count(*) as n
from events
group by device
order by n desc limit 1);
```

	przekroj	dominujaca_wartosc	n
1	event_type	view	799261
2	country	FR	100290
3	device	mobile	333764

Wyniki zapytań są zgodne dla obu baz. Kod jest dokładnie taki sam w obu przypadkach. Najczęstszym zdarzeniem jest wyświetlenie. Jeśli chodzi o pozostałe przekroje, kategorią dominującą dla krajów jest Francja, a dla urządzeń sprzęt mobilny, chociaż ilości zdarzeń dla krajów i urządzeń są do siebie bardzo zbliżone.

3. Aktywność w czasie

PostgreSQL

```
select count(*), date(event_time)
from events
group by date(event_time);
```

	count	date
1	5526	2025-06-15
2	5427	2025-02-26
3	5548	2025-06-16
4	5633	2025-03-30
5	5649	2025-06-13
6	5625	2025-02-11
7	5524	2025-03-28
8	5525	2025-02-17
9	5559	2025-01-09
10	5528	2025-03-27
11	5597	2025-06-18
12	5553	2025-05-02
13	5517	2025-05-25
14	5407	2025-03-23
15	5500	2025-04-06
16	5596	2025-04-29
17	5600	2025-04-14
18	5585	2025-03-24
19	5501	2025-04-25
20	5583	2025-04-27

```sql
select count(*), date(event_time)
from events
group by date(event_time)
order by count(*) desc limit 5;
```

Output Result 20 Result 21 Resu

	count	date
1	5727	2025-02-21
2	5703	2025-02-04
3	5693	2025-04-23
4	5693	2025-02-24
5	5691	2025-02-20

```
select count(*), date(event_time)
from events
group by date(event_time)
order by count(*) asc limit 5;
```

Output | Result 20 | Result 21 | Result 22

	count	date
1	5315	2025-05-01
2	5342	2025-01-11
3	5382	2025-01-29
4	5397	2025-02-02
5	5407	2025-03-23

```
select min(p.count) as min, max(p.count) as max, max(p.count) - min(p.count) as
diff
from (select count(*) as count
      from events
      group by date(event_time)) as p
```

Output | Result 23 | Result 21 | Result 22

	min	max	diff
1	5315	5727	412

ClickHouse

```
select count(*), toDate(event_time)
from events
group by toDate(event_time);
```

	count()	toDate(event_time)
1	5499	2025-01-01
2	5470	2025-01-02
3	5617	2025-01-03
4	5468	2025-01-04
5	5573	2025-01-05
6	5512	2025-01-06
7	5443	2025-01-07
8	5577	2025-01-08
9	5559	2025-01-09
10	5587	2025-01-10
11	5342	2025-01-11
12	5594	2025-01-12
13	5479	2025-01-13
14	5684	2025-01-14
15	5569	2025-01-15
16	5641	2025-01-16
17	5567	2025-01-17
18	5592	2025-01-18
19	5457	2025-01-19
20	5508	2025-01-20
21	5422	2025-01-21
22	5461	2025-01-22
23	5601	2025-01-23
24	5510	2025-01-24
25	5668	2025-01-25
26	5516	2025-01-26
27	5485	2025-01-27

```
select count(*), toDate(event_time)
from events
group by toDate(event_time)
order by count(*) desc limit 5;
```

Output Result 4

	count()	toDate(event_time)
1	5727	2025-02-21
2	5703	2025-02-04
3	5693	2025-02-24
4	5693	2025-04-23
5	5691	2025-02-20

```
select count(*), toDate(event_time)
from events
group by toDate(event_time)
order by count(*) asc limit 5;
```

Output Result 5

	count()	toDate(event_time)
1	5315	2025-05-01
2	5342	2025-01-11
3	5382	2025-01-29
4	5397	2025-02-02
5	5407	2025-03-23

```
select min(p.count) as min, max(p.count) as max, max(p.count) - min(p.count) as
diff
from (select count(*) as count
      from events
      group by toDate(event_time)) as p
```

	min	max	diff
1	5315	5727	412

Wyniki zapytań są takie same dla obu baz. Rozkład wygląda na stabilny, bo różnica miedzy maksymalną liczą zdarzeń a minimalną wynosi zaledwie 412. Nie widać wyraźnych dni odstających.

4. Podstawowe KPI sprzedażowe

Przygotuj podstawowe KPI związane ze zdarzeniami zakupowymi.

ClickHouse

- policz liczbę zdarzeń typu purchase,
- policz łączny przychód ze zdarzeń typu purchase,
- policz średnią wartość pojedynczego zakupu,
- policz średnią liczbę sztuk w pojedynczym zakupie

```
select
    count(*) AS purchases_cnt,
    sum(price * quantity) AS revenue,
    avg(price * quantity) AS avg_order_value,
    avg(quantity) AS avg_quantity
from events
where event_type = 'purchase';
```

	purchases_cnt	revenue	avg_order_value	avg_quantity
1	50333	17449170.359999996	346.6745546659249	1.3757177199849007

- policz liczbę sesji, w których wystąpił co najmniej jeden zakup

```
select
    count(distinct session_id) as purchase_sessions
from events
where event_type = 'purchase'
```

	purchase_sessions
1	44601

- policz udział sesji zakupowych w ogólnej liczbie sesji jako uproszczony współczynnik konwersji

```sql
select
    (select count(distinct session_id) from events where event_type = 'purchase')
* 100.0
    / (select count(distinct session_id) from events) as conv_rate;
```

	conv_rate
1	22.45747776960957

PostgreSQL

- policz liczbę zdarzeń typu purchase,
- policz łączny przychód ze zdarzeń typu purchase,
- policz średnią wartość pojedynczego zakupu,
- policz średnią liczbę sztuk w pojedynczym zakupie

```sql
select
    count(*) AS purchases_cnt,
    sum(price * quantity) AS revenue,
    avg(price * quantity) AS avg_order_value,
    avg(quantity) AS avg_quantity
from events
where event_type = 'purchase';
```

	purchases_cnt	revenue	avg_order_value	avg_quantity
1	50333	17449170.3600001	346.6745546659269	1.3757177199849006

- policz liczbę sesji, w których wystąpił co najmniej jeden zakup

```sql
select
    count(distinct session_id) as purchase_sessions
from events
where event_type = 'purchase'
```

	purchase_sessions
1	44601

- policz udział sesji zakupowych w ogólnej liczbie sesji jako uproszczony współczynnik konwersji

```
select
    (select count(distinct session_id) from events where event_type = 'purchase')
* 100.0
    / (select count(distinct session_id) from events) as conv_rate;
```

	conv_rate
1	22.4574777696095709

Wyniki w obu bazach są zgodne. Udział sesji zakupowych lepiej opisuje konwersję, ponieważ podczas jednej wizyty klient może wygenerowac dziesiątki zdarzeń `view` przeglądając produkty, ale sfinalizować to tylko jednym zdarzeniem `purchase`. Opieranie się na stosunku zdarzeń zaniżałoby wynik, więc w tym przypadku tylko analiza na poziomie sesji odpowiada na pytanie jaki procent wizyt w sklepie faktycznie zakończył się zakupem.

5. KPI w przekrojach biznesowych

PostgreSQL

```
--- wedlug kraju
SELECT
    country,
    sum(price * quantity) AS revenue
FROM events
WHERE event_type = 'purchase'
GROUP BY country
ORDER BY revenue DESC;
```

	country	revenue
1	FR	1805210.3200000061
2	IT	1779036.9600000042
3	PL	1774036.4399999988
4	NL	1753906.4700000002
5	SE	1749563.0099999993
6	GB	1748815.3699999973
7	NO	1733380.0400000021
8	ES	1707084.780000001
9	US	1703552.5699999984
10	DE	1694584.3999999973

```sql
--- wedlug urzadzenia
SELECT
    device,
    sum(price * quantity) AS revenue
FROM events
WHERE event_type = 'purchase'
GROUP BY device
ORDER BY revenue DESC;
```

Output | Result 23 | Result 24 | Result 25

	device	revenue
1	tablet	5838853.980000028
2	desktop	5835272.090000027
3	mobile	5775044.290000002

ClickHouse

Te same polecenia zostału użyte jak dla PostgreSQL.

country	revenue
FR	1805210.3199999991
IT	1779036.9599999995
PL	1774036.4399999997
NL	1753906.4699999993
SE	1749563.0100000019
GB	1748815.37
NO	1733380.0399999998
ES	1707084.7800000005
US	1703552.5700000003
DE	1694584.4000000008

	device	revenue
1	tablet	5838853.980000003
2	desktop	5835272.090000004
3	mobile	5775044.289999999

Wyniki dla obu baz są takie same, różnią się jedynie przybliżeniem obliczeń.

W przekroju krajów Francja generuje najwyższy przychód, natomiast Niemcy najniższy.

W przypadku urządzeń tablet ma najwyższy przychód, przy czym desktop znajduje się na bardzo zbliżonym poziomie. Mobile generuje najniższy przychód.

6. Użytkownicy o najwyższym przychodzie

Znajdź użytkowników o najwyższym łącznym przychodzie z zakupów. Wynik powinien zawierać: user_id, liczbę wszystkich zdarzeń użytkownika, liczbę zdarzeń typu purchase, łączny przychód użytkownika ze zdarzeń typu purchase. Pokaż co najmniej 20 rekordów o najwyższym przychodzie.

ClickHouse

	user_id	all_events	purchase_events	revenue
1	21194	25	4	5037.99
2	11919	23	3	4859.679999999999
3	18860	27	3	4639.92
4	21978	26	3	4604.17
5	39652	27	3	4584.65
6	35735	29	5	4527.26
7	32592	24	4	4526.099999999999
8	48742	21	5	4460.98
9	4935	26	4	4360.120000000001
10	9978	24	3	4301.870000000001
11	33799	22	2	4204.8
12	29614	26	7	4192.2
13	36059	18	4	4160.82
14	7178	25	4	4119.37
15	37756	25	5	4078.3500000000004
16	23725	23	3	4006.11
17	7009	22	3	4001.56
18	3112	24	4	3980.3599999999997
19	47934	27	4	3915.71
20	48299	32	3	3866.2000000000003

PostgreSQL

	user_id	all_events	purchase_events	revenue
1	21194	25	4	5037.99
2	11919	23	3	4859.679999999999
3	18860	27	3	4639.92
4	21978	26	3	4604.17
5	39652	27	3	4584.65
6	35735	29	5	4527.26
7	32592	24	4	4526.1
8	48742	21	5	4460.98
9	4935	26	4	4360.120000000001
10	9978	24	3	4301.87
11	33799	22	2	4204.8
12	29614	26	7	4192.2
13	36059	18	4	4160.820000000001
14	7178	25	4	4119.37
15	37756	25	5	4078.35
16	23725	23	3	4006.1099999999997
17	7009	22	3	4001.56
18	3112	24	4	3980.3599999999997
19	47934	27	4	3915.71
20	48299	32	3	3866.2

Użytkownik z najwyższym przychodem nie ma największej liczby zdarzeń w tym zestawieniu, ma ich 25. Przykładowo, użytkownik na 20 miejscu wygenerował ich aż 32. Widać wyraźnie, że duża aktywność nie jest gwarancją najwyższego zysku np. klient z zaledwie 2 zakupami na pozycji 11 wygenerował wyższy przychód niż klient, który dokonał aż 7 zakupów (pozycja 12). Przychód zależy przede wszystkim od wartości kupowanych produktów, a nie od samej liczby kliknięć czy ilosci transakcji. Klient, który rzadko wchodzi na stronę, ale kupuje drogie produkty, jest z perspektywy przychodu bardziej wartościowy niż bardzo aktywny użytkownik kupujący dużo tanich rzeczy.

7. Benchmark zapytań w PostgreSQL i ClickHouse

- A

```
--- PostgreSQL i ClickHouse
SELECT
    count(*) AS n,
    min(event_time) AS min_time,
    max(event_time) AS max_time
FROM events;
```

Baza	Pomiar 1	Pomiar 2	Pomiar 3	Średnia
PostgreSQL	80	75	75	76.67
ClickHouse	10	8	8	8.67

```
--- PostgreSQL
select min(p.count) as min, max(p.count) as max, max(p.count) - min(p.count) as
diff
from (select count(*) as count
      from events
      group by date(event_time)) as p;
```

```
--- ClickHouse
select min(p.count) as min, max(p.count) as max, max(p.count) - min(p.count) as
diff
from (select count(*) as count
      from events
      group by toDate(event_time)) as p;
```

Baza	Pomiar 1	Pomiar 2	Pomiar 3	Średnia
PostgreSQL	156	153	152	153.67
ClickHouse	32	13	13	19.3

- B

```sql
--- PostgreSQL
SELECT
    DATE(event_time) AS day,
    country,
    device,
    event_type,
    count(*) AS events_cnt,
    count(DISTINCT user_id) AS users_cnt,
    count(DISTINCT session_id) AS sessions_cnt,
    sum(CASE
            WHEN event_type = 'purchase' THEN price * quantity
            ELSE 0
        END) AS revenue
FROM events
where event_type = 'purchase'
GROUP BY
    DATE(event_time),
    country,
    device,
    event_type
ORDER BY revenue DESC
LIMIT 20;
```

	day	country	device	event_type	events_cnt	users_cnt	sessions_cnt	revenue
1	2025-06-13	US	mobile	purchase	14	14	14	10827.559999999998
2	2025-04-21	US	tablet	purchase	21	21	21	9945.939999999997
3	2025-01-23	IT	tablet	purchase	19	19	19	9584.159999999998
4	2025-05-15	IT	desktop	purchase	15	15	15	9468.63
5	2025-05-30	DE	desktop	purchase	18	18	18	9445.289999999999
6	2025-03-04	US	tablet	purchase	18	18	18	9403.07
7	2025-01-04	GB	tablet	purchase	20	20	20	9342.119999999999
8	2025-02-04	FR	tablet	purchase	21	21	21	9139.390000000003
9	2025-02-24	IT	desktop	purchase	16	16	16	9022.180000000002
10	2025-04-19	PL	mobile	purchase	15	15	15	8954.64
11	2025-05-25	DE	mobile	purchase	15	15	15	8909.98
12	2025-06-10	DE	desktop	purchase	21	21	21	8884.72
13	2025-03-24	IT	desktop	purchase	17	17	17	8801.78
14	2025-03-20	IT	desktop	purchase	11	11	11	8796.71
15	2025-05-10	IT	tablet	purchase	15	15	15	8755.74
16	2025-04-19	ES	mobile	purchase	13	13	13	8738.15
17	2025-06-14	PL	tablet	purchase	14	14	14	8689.470000000001
18	2025-04-26	US	tablet	purchase	14	14	14	8663.769999999999
19	2025-03-25	SE	mobile	purchase	15	15	15	8570.769999999999
20	2025-04-18	FR	mobile	purchase	13	13	13	8570.720000000001

```sql
--- ClickHouse
SELECT
    toDate(event_time) AS day,
    country,
    device,
    event_type,
    count() AS events_cnt,
    count(DISTINCT user_id) AS users_cnt,
    count(DISTINCT session_id) AS sessions_cnt,
```

```
    sum(CASE
            WHEN event_type = 'purchase' THEN price * quantity
            ELSE 0
        END) AS revenue
FROM events
GROUP BY
    day,
    country,
    device,
    event_type
ORDER BY revenue DESC
LIMIT 20;
```

Output | Result 1 | CSV

	day	country	device	event_type	events_cnt	users_cnt	sessions_cnt	revenue
1	2025-06-13	US	mobile	purchase	14	14	14	10827.56
2	2025-04-21	US	tablet	purchase	21	21	21	9945.939999999999
3	2025-01-23	IT	tablet	purchase	19	19	19	9584.16
4	2025-05-15	IT	desktop	purchase	15	15	15	9468.630000000001
5	2025-05-30	DE	desktop	purchase	18	18	18	9445.29
6	2025-03-04	US	tablet	purchase	18	18	18	9403.070000000002
7	2025-01-04	GB	tablet	purchase	20	20	20	9342.12
8	2025-02-04	FR	tablet	purchase	21	21	21	9139.39
9	2025-02-24	IT	desktop	purchase	16	16	16	9022.180000000002
10	2025-04-19	PL	mobile	purchase	15	15	15	8954.640000000001
11	2025-05-25	DE	mobile	purchase	15	15	15	8909.98
12	2025-06-10	DE	desktop	purchase	21	21	21	8884.72
13	2025-03-24	IT	desktop	purchase	17	17	17	8801.78
14	2025-03-20	IT	desktop	purchase	11	11	11	8796.710000000001
15	2025-05-10	IT	tablet	purchase	15	15	15	8755.74
16	2025-04-19	ES	mobile	purchase	13	13	13	8738.15
17	2025-06-14	PL	tablet	purchase	14	14	14	8689.470000000001
18	2025-04-26	US	tablet	purchase	14	14	14	8663.77
19	2025-03-25	SE	mobile	purchase	15	15	15	8570.77
20	2025-04-18	FR	mobile	purchase	13	13	13	8570.720000000001

Baza	Pomiar 1	Pomiar 2	Pomiar 3	Średnia
PostgreSQL	167	144	150	153.67
ClickHouse	80	66	59	68.3

- C

```
--- PostgreSQL
SELECT
    DATE(event_time) AS day,
    country,
    event_type,
    COUNT(*) AS events_cnt,
    COUNT(DISTINCT user_id) AS users_cnt,
    COUNT(DISTINCT session_id) AS sessions_cnt,
    SUM(price * quantity) AS revenue
FROM events
WHERE event_type = 'purchase'
```

```sql
  AND country = 'US'
  AND event_time >= NOW() - INTERVAL '300 days'
GROUP BY
    DATE(event_time),
    country,
    event_type
ORDER BY revenue DESC
LIMIT 30;
```

Output | Result 7

	day	country	event_type	events_cnt	users_cnt	sessions_cnt	revenue
1	2025-06-28	US	purchase	30	30	30	10328.36
2	2025-06-26	US	purchase	24	24	24	9912.060000000001
3	2025-06-27	US	purchase	31	31	31	8206.300000000001
4	2025-06-29	US	purchase	25	25	25	8031.01
5	2025-06-25	US	purchase	1	1	1	380.11

```sql
--- ClickHouse
SELECT
    toDate(event_time) AS day,
    country,
    event_type,
    count() AS events_cnt,
    countDistinct(user_id) AS users_cnt,
    countDistinct(session_id) AS sessions_cnt,
    sum(price * quantity) AS revenue
FROM events
WHERE event_type = 'purchase'
  AND country = 'US'
  AND event_time >= now() - INTERVAL 300 DAY
GROUP BY
    day,
    country,
    event_type
ORDER BY revenue DESC
LIMIT 30;
```

Output | Result 24

	day	country	event_type	events_cnt	users_cnt	sessions_cnt	revenue
1	2025-06-28	US	purchase	30	30	30	10328.359999999995
2	2025-06-26	US	purchase	24	24	24	9912.06
3	2025-06-27	US	purchase	31	31	31	8206.300000000001
4	2025-06-29	US	purchase	25	25	25	8031.01
5	2025-06-25	US	purchase	2	2	2	877.3199999999999

Baza	Pomiar 1	Pomiar 2	Pomiar 3	Średnia
PostgreSQL	80	78	92	83.3
ClickHouse	14	12	13	13

Wyniki wszystkich zapytań były zgodne między PostgreSQL a ClickHouse (jedynie z drobnymi różnicami wynikającymi z zaokrągleń). ClickHouse był konsekwentnie szybszy we wszystkich przypadkach z wyraźnymi różnicami w czasie wykonania. Największa różnica wystąpiła w zapytaniu A dla drugiego wybranego przykładu, gdzie PostgreSQL potrzebował średnio 153.67 ms wobec 19.3 ms w ClickHouse. Po tym ćwiczeniu można postawić wstępny wniosek, że ClickHouse jest znacznie lepiej przystosowany do agregacji analitycznych na dużych zbiorach danych. Jego kolumnowy model przechowywania danych pozwala przetwarzać tylko niezbędne kolumny, co przekłada się na wyraźną przewagę wydajnościową szczególnie przy złożonych zapytaniach agregujących.